Midterm Exam Study Guide

12 Topics • Plain-English Explanations • Examples • Exam-Answer Structure

Quantum Computing • Blockchain • Cyber Security • LLMs in Software Engineering • LLM & VLLM in Data Science • Computer Vision & Generative AI • Ontologies & Data Mining • Knowledge Representation & Frames • Semantic Networks, Rules & Semantic Web • CBR & Constraints • NLP • Quantum Computing vs Generative AI

Built so all topics can be covered in one day. Each topic: simple explanation → key points → example/scenario → how to write it in the exam.

One-Day Study Plan

Suggested schedule

Time	Topics	Focus
Morning (90 min)	1. Quantum Computing, 12. Quantum vs GenAI	Qubit, Grover, Shor, comparison table
(60 min)	2. Blockchain, 3. Cyber Security	Block structure, consensus, CIA triad, attacks
Afternoon (90 min)	4, 5, 6, 11 (LLMs, VLLM, Computer Vision, NLP)	Use cases, pipelines, GAN/Diffusion, Transformer
Late afternoon (90 min)	7, 8, 9 (Ontology, KR & Frames, Semantic Nets, Rules, Semantic Web)	Definitions + worked examples are asked often
Evening (45 min)	10. CBR & Constraints	4R cycle, map coloring, backtracking
Night (60 min)	Revision	Only the *Exam Tip* boxes and the final Cheat Sheet

General formula for any exam answer

1) Definition (1–2 lines) → 2) Key components/steps (as points) → 3) One real-life example → 4) Advantages/limitations → 5) Diagram or table where possible. Following these five steps gets you close to full marks on almost any question.

Note

"VLLM" in Topic 5 is covered both ways: (a) Vision-Language Large Models (understand images + text) and (b) the vLLM inference engine. Keep whichever your course used.

1. Quantum Computing

Principles • Qubits • Grover's • Shor's

1.1 Core Principles

Quantum computing performs computation using the rules of quantum mechanics: superposition, entanglement and interference. A classical **bit** is either 0 or 1; a quantum **qubit** can be a blend of 0 and 1 until it is measured.

Principle	Simple explanation
Superposition	A qubit is a combination of $\|0\rangle$ and $\|1\rangle$ at once. Like a spinning coin that is "both" head and tail as probabilities.
Entanglement	Two qubits are linked so that knowing one result immediately tells you the other, however far apart they are.
Interference	Wrong answers' amplitudes are cancelled and correct answers' amplitudes are boosted. This is the real source of quantum algorithms' power.
Measurement	Measuring collapses the superposition into a definite 0 or 1.

1.2 The Qubit

State of one qubit: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α, β are complex numbers and $|\alpha|^2 + |\beta|^2 = 1$. Measuring gives 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$.

- **Bloch sphere:** a qubit state is a point on a sphere. North pole = $|0\rangle$, south pole = $|1\rangle$.
- **n qubits** hold amplitudes for 2^n states at once. 3 qubits = 8 states; 50 qubits ≈ 10^{15} states.
- **Gates:** X (NOT, flips $0 \leftrightarrow 1$), H (Hadamard, creates superposition), Z (phase flip), CNOT (2-qubit gate, creates entanglement).
- **Physical qubits:** superconducting circuits, trapped ions, photons. Noise/decoherence is the big problem, hence error correction.

> **Example: Bell state (entanglement)**
>
> Start with $|00\rangle$ → apply H to the first qubit → apply CNOT. Result: $(|00\rangle + |11\rangle)/\sqrt{2}$. Measuring gives 00 half the time and 11 half the time, never 01 or 10. If one qubit reads 0, the other is certain to read 0. That is entanglement.

1.3 Grover's Algorithm

Purpose: search an unsorted database for a target item. Classically, N items need about N/2 tries on average (N worst case). Grover needs only **O(√N)** tries: a **quadratic speedup** (not exponential).

1. **Initialize:** apply H to all qubits to get an equal superposition of all N states.
2. **Oracle:** flips the sign (phase) of the correct answer's amplitude.
3. **Diffusion operator (amplitude amplification):** reflects all amplitudes about their average, which raises the correct answer's amplitude.
4. Repeat steps 2–3 about $(\pi/4)\cdot\sqrt{N}$ times, then measure.

Example (N = 4, 2 qubits)

Four items (00, 01, 10, 11), target = 10. Initially each has 25% probability. After just **1 iteration** the probability of 10 is **100%**. Classically you'd check 2–3 items on average.

Real scenario: matching one hash among 1,000,000 password candidates: classical ≈ 10^6 steps, Grover ≈ 10^3 steps.

Limitations: only quadratic speedup; you must be able to build the oracle; not useful for every problem. Effect on symmetric crypto: double the key length (AES-128 → AES-256).

1.4 Shor's Algorithm

Purpose: factor large integers into primes. The best classical algorithms are sub-exponential; Shor's runs in **polynomial time** (about $O((\log N)^3)$). This threatens **RSA, Diffie-Hellman and ECC**.

Key idea: turn factoring into **period finding**. Period finding is the quantum part (using the **Quantum Fourier Transform**); the rest is classical.

1. Pick a with $1 < a < N$ that is coprime to N.
2. Find the **period r** of $f(x) = a^x \bmod N$ (quantum part, QFT).
3. If r is even and $a^{r/2} \not\equiv -1 \pmod N$, then $\gcd(a^{r/2} \pm 1, N)$ gives the factors.
4. Otherwise try another a.

Example: factor N = 15

Take a = 7. $7^1=7$, $7^2=49 \bmod 15=4$, $7^3=28 \bmod 15=13$, $7^4=91 \bmod 15=1$ → **period r = 4** (even).
$a^{r/2} = 7^2 = 49$. $\gcd(49-1, 15) = \gcd(48, 15) = \mathbf{3}$; $\gcd(49+1, 15) = \gcd(50, 15) = \mathbf{5}$. So $15 = 3 \times 5$.
The quantum computer's only job is finding r = 4 quickly. For a 2048-bit number this is impossible classically.

	Grover	Shor
Problem	Unstructured search	Integer factoring / discrete log
Speedup	Quadratic (N → √N)	Exponential (super-polynomial)
Main technique	Amplitude amplification	Period finding + QFT
Impact	Weakens symmetric keys (doubling key size suffices)	Breaks RSA/ECC → Post-Quantum Cryptography needed

Exam Tip

"Explain Grover's algorithm": purpose → 3 steps (superposition, oracle, diffusion) → O(√N) → N=4 example. "Shor's": factoring → period finding → N=15, a=7 example → impact on RSA. A Grover-vs-Shor table earns extra marks.

2. Blockchain

Distributed ledger • Consensus • Smart contracts

2.1 Definition

Blockchain is a **distributed, decentralized, immutable ledger**. Transactions are stored in **blocks**, and each block is cryptographically linked to the previous block's **hash**. No single authority (such as a bank) is needed.

2.2 Structure of a Block

Field	Role
Previous hash	Fingerprint of the previous block; this is what forms the "chain"
Merkle root	Root of a hash tree of all transactions; changing any transaction changes the root
Timestamp, Nonce	Time; Nonce = number varied in Proof-of-Work to find a valid hash
Transactions	The actual data (who sent what to whom)
Block hash	SHA-256 hash of the whole header

Why can't it be tampered with?
Change data in Block 5 → its hash changes → Block 6's "previous hash" no longer matches → Blocks 6, 7, 8… must all be recomputed, and a majority of network nodes must be convinced. Practically impossible.

2.3 Consensus Mechanisms

Type	How it works	Pros / Cons
Proof of Work (PoW)	Miners solve a hash puzzle (find a nonce so the hash starts with a set number of zeros). The first to solve it adds the block and earns a reward. Used by Bitcoin.	Very secure / huge electricity use, slow (~7 TPS)
Proof of Stake (PoS)	Validators are chosen to validate blocks based on the coins they "stake". Cheating loses the stake. Used by Ethereum now.	Low energy, faster / wealthy validators have more influence
PBFT / others	Nodes in a permissioned network vote	Fast / limited number of nodes

2.4 Types and Smart Contracts

- **Public** (Bitcoin, Ethereum): anyone can join. **Private/Consortium** (Hyperledger Fabric): only approved members; used by companies.
- **Smart contract:** self-executing code stored on the blockchain. It runs automatically when conditions are met, with no third party.
- **Key properties:** decentralization, transparency, immutability, security (cryptography), traceability.

Example 1: Land registry
Every ownership change is recorded on the blockchain. Double-selling the same plot or forged deeds are easy to catch because past records cannot be altered. Smart contract: "When the buyer pays, ownership automatically transfers to them."
Example 2: Supply chain: mango farm → packing → transport → shop; each step is scanned and recorded. A customer scans a QR code and sees where the mango came from.

Challenges: scalability (low TPS), energy use (PoW), 51% attack (someone controls over half the network's computing power), privacy, regulation.

Exam Tip
Draw the diagram: [Block 1 | hash H1] → [Block 2 | prev H1, hash H2] → [Block 3 | prev H2, …]. That alone secures about half the marks. Then add the PoW vs PoS table.

3. Cyber Security

CIA triad • Threats • Defense

3.1 Definition and the CIA Triad

Cyber security is the practice of protecting computers, networks, data and systems from unauthorized access, attacks and damage.

Principle	Meaning	Example threat	Protection
Confidentiality	Only authorized people see the data	Data breach, sniffing	Encryption, access control
Integrity	Data stays unaltered	Data tampering, MITM	Hashing, digital signatures
Availability	Service is available when needed	DDoS, ransomware	Backups, redundancy, firewall

3.2 Major Threats and Attacks

Attack	What it is	Example
Malware (virus, worm, trojan, ransomware, spyware)	Malicious software	Ransomware encrypts files and demands payment
Phishing	Fake emails/sites that steal passwords or money	"Your bank account will be closed, log in here" link
SQL Injection	Injecting SQL code into an input field to control the database	Entering `' OR '1'='1` in a password field bypasses login
XSS	Injecting a malicious script into a website	A comment containing a script steals other users' cookies
DoS / DDoS	Flooding a server with requests until it fails	Thousands of bots hit one site at once
Man-in-the-Middle	Sitting between two parties to listen or modify traffic	Capturing data on public WiFi
Social engineering	Tricking people into giving information	Phoning while pretending to be IT staff and asking for a password

3.3 Defense Tools

- **Cryptography:** symmetric (AES, same key) is fast; asymmetric (RSA, public + private key) for key exchange; hashing (SHA-256) for integrity; digital signature = hash + private key.
- **Authentication:** password + **MFA/2FA** (OTP, biometrics).
- **Firewall** (filters traffic), **IDS/IPS** (detect/prevent attacks), **VPN**, **antivirus**, regular **patching**.
- **Defense in depth:** security at multiple layers (network → host → application → data → people/training).
- **Prevention:** parameterized queries against SQLi; input sanitization against XSS.

> **Scenario: banking phishing**
>
> Rahim receives an SMS: "Your card is blocked, go to bkash-bank-verify.xyz". He enters his card number and OTP on the fake page and the attacker withdraws his money. **Violated:** Confidentiality. **Prevention:** check the URL, never share OTPs, use MFA, use the bank's official app, user awareness training.

Exam Tip

Always write the CIA triad. For each attack use four steps: what it is → how it works → one example → prevention. Writing the SQL injection query example makes a strong impression.

4. LLMs in Software Engineering

Code generation • Testing • Review • Risks

4.1 What is an LLM?

A **Large Language Model (LLM)** is a huge Transformer-based neural network trained on vast amounts of text and code. It predicts the next token to generate text or code. Examples: GPT, Claude, Gemini, Llama, Code Llama, StarCoder.

4.2 Use Across the SDLC

Phase	What the LLM does
Requirements	Turns a client's messy description into user stories or an SRS draft; flags ambiguity
Design	Architecture suggestions, UML/ER diagram descriptions, API design
Coding	Code generation, auto-completion (GitHub Copilot), translation between languages
Testing	Generates unit tests, finds edge cases, creates test data
Debugging	Explains error messages, proposes bugs and fixes, **automated program repair**
Review & Docs	Code review comments, docstrings/README, commit messages
Maintenance	Explains legacy code, refactoring, detects code smells

4.3 Key Techniques

- **Prompt engineering:** clear context, examples (few-shot), step-by-step reasoning (chain-of-thought).
- **RAG (Retrieval-Augmented Generation):** retrieve relevant parts of your codebase/docs and put them in the prompt, which reduces hallucination.
- **Fine-tuning:** adapt to a company's own code style or domain.
- **Agents:** the LLM runs tools itself (edit files, run tests, use git) and fixes its own mistakes.
- **Evaluation:** HumanEval and MBPP benchmarks; metric **pass@k** = probability that at least one of k attempts is correct code.

Example: test generation

Prompt: "Write pytest unit tests including edge cases for this function." Function: `def divide(a,b): return a/b`. The LLM writes tests for normal (10/2), negative, float, and **b=0 expecting ZeroDivisionError**. An hour of developer work takes seconds, but a human must still review it.

4.4 Risks and Limitations

- **Hallucination:** invents functions or libraries that do not exist.
- **Security:** vulnerable code (SQLi, hard-coded secrets), prompt injection.
- **License/IP:** copyrighted code from training data may appear verbatim.
- **Over-reliance:** developer skills decline; code merged without understanding.
- **Privacy:** sending proprietary code to an external API.

- Context window limits on large projects.

> **Exam Tip**
> Answer with an SDLC-phase table + one example (test generation or bug fix) + benefits vs risks in two columns. Conclude: "LLMs do not replace developers; they boost productivity, and a human in the loop is essential."

5. LLM & VLLM in Data Science

Text-to-SQL • EDA • Vision-Language Models • vLLM serving

5.1 LLMs in the Data Science Pipeline

Stage	Role of the LLM
Data collection	Extract structured fields from unstructured text (reviews, PDFs, emails): information extraction
Cleaning	Fix spelling/format, guess missing values, standardize categories ("Dhaka", "DHK" → Dhaka)
EDA	Ask questions in natural language; it writes pandas/SQL code and charts
Feature engineering	Text embeddings, ideas for new features
Modeling	Model suggestions, hyperparameters, boilerplate code
Reporting	Explains results, summaries, insight narrative

> **Example: Text-to-SQL**
> Question: "Which 5 products sold the most last month?" → the LLM reads the schema and writes:
>
> ```
> SELECT product_name, SUM(qty) AS total
> FROM sales
> WHERE sale_date >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
> GROUP BY product_name ORDER BY total DESC LIMIT 5;
> ```
>
> A business user gets answers without knowing SQL. **Caution:** verify before running (wrong JOIN, wrong column).

5.2 VLLM: Two Meanings

(a) Vision-Language (Large) Models, VLM

These models understand **images + text** together. Architecture: vision encoder (ViT/CLIP) → projection layer → LLM. Examples: GPT-4V/4o, Gemini, Claude (vision), LLaVA, CLIP.

- **Reading charts/graphs:** given a dashboard image, explains the trend.
- **Document/OCR:** extract data from images of invoices, forms and tables.
- **Medical/satellite images:** description and Visual Question Answering.
- **Image captioning, image-text retrieval**, multimodal dataset labeling.

Scenario

A company has thousands of scanned or handwritten bill images. The VLM is told: "From this image give date, vendor, total as JSON." The result is a CSV without manual data entry, ready for ordinary data science analysis.

(b) vLLM: an inference/serving engine

vLLM is an open-source library that serves LLMs faster and with less memory. Its key innovation is **PagedAttention**: the KV cache is split into small blocks like OS virtual-memory paging, reducing GPU memory waste. Together with **continuous batching** it gives higher throughput. Useful when a data science team runs its own LLM in production.

Exam Tip

When a question says "VLLM", first decide which meaning (from the question's context). If unsure, write both briefly. Always mention LLM limitations (hallucination, data privacy, wrong SQL).

6. Computer Vision Algorithms & Generative AI

Classical CV • CNN • Detection • GAN • Diffusion

6.1 What is Computer Vision (CV)?

Teaching computers to extract meaning from images and video. Main tasks: **classification** (what is in the image), **detection** (where, via bounding boxes), **segmentation** (pixel-level), tracking, pose estimation, OCR.

6.2 Classical Algorithms

Algorithm	Use
Sobel / **Canny**	Edge detection (Canny: blur → gradient → non-max suppression → thresholding)
Harris corner	Corner detection
SIFT / SURF / ORB	Scale/rotation-invariant features, image matching, panorama stitching
HOG + SVM	Pedestrian detection
Hough transform	Detect lines/circles
K-means, Watershed	Segmentation

6.3 Deep Learning

- **CNN:** convolution layers (filters detect edge → shape → object) → pooling (downsample) → fully connected → softmax. Examples: LeNet, AlexNet, VGG, **ResNet** (skip connections fix the vanishing-gradient problem).
- **Object detection:** R-CNN/Faster R-CNN (two-stage, accurate), **YOLO** / SSD (one-stage, real-time).
- **Segmentation:** U-Net (medical), Mask R-CNN.
- **Vision Transformer (ViT):** split the image into patches and feed them to a Transformer.

Example: self-driving car

Camera frame → YOLO detects cars/people/traffic lights → U-Net segments the road → lane detection (Canny + Hough) → decision: brake/turn.

6.4 Generative AI in Vision

Model	Idea	Pros / Cons
GAN	Two networks compete: the **Generator** makes fake images, the **Discriminator** tells real from fake. Both improve. (Counterfeiter vs police.)	Sharp images / unstable training, mode collapse
VAE	Encoder maps the image to a latent space; decoder generates an image from it	Stable / somewhat blurry images
Diffusion	Forward: add noise to an image step by step. Reverse: remove noise step by step to create an image (guided by a text prompt). Stable Diffusion and DALL·E work this way.	Best quality and diversity / slow (many steps)

Use cases: text-to-image, inpainting, super-resolution, style transfer, data augmentation (synthetic medical images when data is scarce), deepfakes (a risk).

> **Exam Tip**
> Draw the GAN generator/discriminator diagram. Write the CNN layer flow. "Discriminative vs generative" is also asked: discriminative models learn P(y|x) (classify); generative models learn P(x) (create new data).

7. Ontologies & Data Mining

Concepts • OWL • KDD • Apriori • Clustering

7.1 Ontology

An **ontology** is a **formal, explicit, shared specification** of the concepts in a domain, their relationships and rules (Gruber: "explicit specification of a conceptualization"). Simply: a "vocabulary + map of relationships" that both machines and people understand.

Component	Meaning	Example (University)
Classes (Concepts)	Categories of things	Person, Student, Professor, Course
Instances (Individuals)	Specific things	Rahim (Student), CSE101 (Course)
Properties (Relations)	Relationships/attributes	enrolledIn, teaches, hasEmail
Axioms / Rules	Constraints	Student $\sqsubseteq$ Person; a Student is enrolled in at least 1 Course

Languages: RDF → RDFS → **OWL** (Web Ontology Language); tool: Protégé. A reasoner (HermiT) infers new knowledge. **Uses:** medicine (SNOMED, Gene Ontology), e-commerce product taxonomies, knowledge graphs.

> **Reasoning example**
> Ontology: Professor $\sqsubseteq$ Person; Rahim is a Professor. Nobody ever wrote "Rahim is a Person", but the reasoner infers Rahim $\in$ Person.

7.2 Data Mining

Data mining discovers previously unknown, useful patterns and knowledge in large data. It is the core step of the **KDD** (Knowledge Discovery in Databases) process: Selection → Preprocessing/Cleaning → Transformation → **Data Mining** → Interpretation/Evaluation.

Task	Goal	Algorithms	Example
Classification	Predict a label (supervised)	Decision Tree, Naive Bayes, SVM, kNN	Loan approve/reject
Clustering	Find similar groups (unsupervised)	K-means, DBSCAN, Hierarchical	Customer segmentation
Association rules	Find items that occur together	Apriori, FP-Growth	Market basket
Regression	Predict a number	Linear regression	House price
Anomaly detection	Spot outliers	Isolation Forest	Credit card fraud

Apriori numeric example

Five transactions: T1{bread, milk}, T2{bread, diaper, beer}, T3{milk, diaper, beer, cola}, T4{bread, milk, diaper, beer}, T5{bread, milk, diaper, cola}.
Rule **{diaper}** → **{beer}**: diaper appears in 4 (T2, T3, T4, T5); diaper+beer in 3 (T2, T3, T4).
Support = 3/5 = 60%. **Confidence** = 3/4 = 75%. **Lift** = confidence / P(beer) = 0.75 / 0.6 = 1.25 (>1 means a positive association).
Support = how often the items occur together; Confidence = probability of B given A.

7.3 Ontology + Data Mining

An ontology makes data mining **semantic**: it resolves whether "Apple" is a fruit or a company, and lets results be generalized through concept hierarchies (Coke, Pepsi → Soft drink) for multi-level association rules. In the other direction, data mining can learn new concepts for an ontology (**ontology learning**).

Exam Tip

For ontology, draw a small university or hospital ontology (class–subclass arrows). For data mining, show the KDD flow plus an Apriori support/confidence calculation for full marks.

8. Knowledge Representation, Frame-Based Systems & Representations for KBS

KR schemes • Frames • Slots • Inheritance

8.1 Knowledge Representation (KR)

KR is organizing real-world knowledge so a computer can store it and reason over it to derive new conclusions. A **KBS (Knowledge-Based System)** = knowledge base + inference engine (+ user interface).

Four properties of a good KR

Property	Meaning
Representational Adequacy	Can express all the kinds of knowledge needed
Inferential Adequacy	Can derive new knowledge from existing knowledge
Inferential Efficiency	Reasoning is fast and resource-light
Acquisitional Efficiency	New knowledge is easy to add

8.2 Main KR Approaches

Approach	In brief	Example
Propositional / Predicate Logic	Precise knowledge as formulas	∀x Student(x) → Person(x)
Production Rules	IF–THEN	IF fever AND cough THEN flu possible
Semantic Networks	Graph of nodes and labeled arcs	Bird –is-a→ Animal
Frames	Structured record of an object	Detailed below
Scripts	Stereotyped sequence of events	Restaurant: enter → order → eat → pay → leave
Ontologies / Description Logic	Formal concept hierarchy	OWL

8.3 Frame-Based Systems

A **frame** (Marvin Minsky, 1975) is a data structure holding stereotypical knowledge about an object or concept. It has **slots** (attributes), and each slot has **facets** (value, default, range/type, procedure).

- **Inheritance:** a child frame inherits slots from its parent frame (via `is-a` and `instance-of` links).
- **Default values:** assumed when nothing else is known (Bird.can_fly = true); a child can override them (Penguin.can_fly = false).
- **Procedural attachment (demons):** *if-needed* (compute on demand), *if-added* (trigger when a value is set), *if-removed*.

Example: Vehicle frame hierarchy

```
Frame: Vehicle
  wheels: 4 (default)
  fuel: petrol (default)

Frame: Car        (is-a Vehicle)
  seats: 5
  max_speed: if-needed -> calculate_from(engine_cc)

Frame: MyToyota   (instance-of Car)
  color: white
  engine_cc: 1500
  owner: Karim
```

Query: how many wheels does MyToyota have? Not in MyToyota → not in Car → inherited from Vehicle: **4**. If max_speed is requested, the if-needed demon computes it from engine_cc.

Advantages	Disadvantages
Natural organization, less redundancy through inheritance, default reasoning, procedures can be attached	Exception handling is hard (multiple-inheritance conflicts), no strict formal semantics, weak for complex logical reasoning

8.4 Representations for KBS

Choose the representation to fit the problem: **Rule-based** (diagnosis, expert systems such as MYCIN), **Frame-based** (structured object knowledge, such as product or patient records), **Logic** (certain proofs), **Semantic nets/Ontologies** (concept relationships). Real systems are often **hybrid** (frames + rules).

> **Exam Tip**
> For frames, always draw a hierarchy example and answer one query by inheritance. Memorize the 4 KR properties (RA, IA, IE, AE).

9. Semantic Networks, Rule-Based Systems & Semantic Web

Graph knowledge • Chaining • RDF/OWL/SPARQL

9.1 Semantic Network

Knowledge as a **graph**: **nodes** = concepts/objects, **labeled arcs** = relationships. Main links: `is-a` (class), `has-a / part-of`, `instance-of`.

> **Example**
>
> ```
> Tweety --instance-of--> Canary --is-a--> Bird --is-a--> Animal
> Bird --has--> Wings        Bird --can--> Fly
> Canary --color--> Yellow
> ```
>
> Question: can Tweety fly? Walk up the graph to Bird, which says "can fly" → **yes** (property inheritance).
> **Limits:** exceptions (Penguin) and quantifiers (all/some) are hard to express; no standard semantics.

9.2 Rule-Based System

Knowledge as **IF (condition) THEN (action/conclusion)**. The core of expert systems.

Part	Role
Knowledge base (rules)	All the rules
Working memory	Current facts
Inference engine	Matches rules to derive conclusions
Explanation facility	Answers "why this conclusion?"

Forward chaining (data-driven)	Backward chaining (goal-driven)
Start from facts → fire rules to derive new facts → reach the goal	Start from a goal (hypothesis) → find which rule proves it → prove that rule's conditions
Good for monitoring, planning	Good for diagnosis (MYCIN)

Example: chaining

Rules: R1: IF A AND B THEN C. R2: IF C THEN D. R3: IF D AND E THEN F. Facts: A, B, E. Goal: F?
Forward: A,B → C (R1) → D (R2) → D,E → F (R3) ✔.
Backward: want F → R3 needs D and E → E is a fact ✔; want D → R2 needs C → R1 needs A,B ✔ → F proven.
Conflict resolution (when several rules can fire): priority, specificity (more specific rule first), recency.

Advantages: easy to understand, modular, can explain itself. **Disadvantages:** hard to maintain with many rules, poor at uncertainty (needs certainty factors/fuzzy logic), no learning.

9.3 Semantic Web

Tim Berners-Lee's idea: give web data machine-understandable meaning so software can connect and understand data itself. The current web is for people; the Semantic Web is for machines too.

- **URI/IRI:** a unique identity for every thing.
- **RDF:** data as **triples**: **Subject – Predicate – Object**.
- **RDFS:** class and property hierarchies. **OWL:** richer ontologies (cardinality, disjointness, inverse properties).
- **SPARQL:** query language for RDF. **Linked Data**, DBpedia, schema.org.

RDF triples + SPARQL

```
:Rahim   :studiesAt   :DhakaUniversity .
:Rahim   :age         22 .
:DhakaUniversity :locatedIn :Dhaka .

SELECT ?name WHERE {
  ?name :studiesAt ?u .
  ?u    :locatedIn :Dhaka .
}
```

Answer: Rahim. The answer comes from joining two separate triples in the graph.

Layer cake (bottom to top): Unicode/URI → XML → RDF → RDFS → Ontology (OWL) → Rules (SWRL) → Logic → Proof → Trust.

Exam Tip

For chaining, show both forward and backward on a small rule set, step by step. For the Semantic Web, drawing the layer cake and writing one RDF triple almost always earns marks.

10. Case-Based Reasoning & Reasoning with Constraints

4R cycle • Similarity • CSP • Backtracking

10.1 Case-Based Reasoning (CBR)

Solving a new problem by reusing the **solution of a similar past problem**, like a doctor drawing on earlier patients. A **case** = problem description + solution (+ outcome).

The CBR Cycle (4R)

1. **Retrieve:** find the most similar case in the case base (similarity measure, e.g. nearest neighbor).
2. **Reuse:** apply that case's solution to the new problem.

3. **Revise:** adapt or correct the solution if needed, and verify it.
4. **Retain:** store the successful new case in the case base (learning).

Similarity: $Sim(new, old) = \Sigma\ w_i \cdot sim_i(feature_i)$ (weighted sum). Weights give important features more influence.

> **Example: IT help desk**
> New problem: "Laptop will not connect to WiFi, Windows 11, Intel adapter". An earlier case: "WiFi not working, Windows 11, Intel adapter → fixed by driver update" (similarity 92%). **Retrieve** it, **Reuse**: tell the user to update the driver. If that fails, **Revise**: add an adapter reset. When it works, **Retain** the new case.

Advantages: no need to write expert rules, learns gradually, easy to explain. **Disadvantages:** retrieval slows as the case base grows, good similarity/adaptation is hard to build, weak on completely new problems. **Uses:** help desks, law (precedent), medical diagnosis, recipe planning.

10.2 Reasoning with Constraints (CSP)

A **Constraint Satisfaction Problem** = **Variables (X)** + each variable's **Domain (D)** + **Constraints (C)**. Goal: assign a value to every variable so all constraints hold.

> **Example: map coloring (Australia)**
> Variables: WA, NT, SA, Q, NSW, V, T. Domain: {Red, Green, Blue}. Constraint: neighboring regions get different colors (WA≠NT, WA≠SA, NT≠SA, SA≠Q, …).
> Solution: WA=Red, NT=Green, SA=Blue, Q=Red, NSW=Green, V=Red, T=any.
> **Other examples:** Sudoku, N-Queens, class/exam timetabling (no two classes in the same room at the same time).

Solution techniques

- **Backtracking search:** assign one variable at a time; if a constraint is violated, step back and try another value.
- **Forward checking:** after an assignment, remove inconsistent values from neighbors' domains. If any domain becomes empty, backtrack immediately.
- **Arc consistency (AC-3):** for each constraint, make sure every value on one side has a compatible value on the other; otherwise delete it.
- **Heuristics: MRV** (variable with the fewest legal values first), **Degree** (variable in the most constraints first), **LCV** (try the value that rules out the fewest neighbor options first).

> **Forward checking (small example)**
> A, B, C must all differ, domain {1,2}. Set A=1 → B and C domains become {2}. Set B=2 → C's domain becomes empty {} → fail immediately and backtrack. (This problem has no solution: three different values are needed but only two exist.)

> **Exam Tip**
> For CBR, draw the circular 4R diagram. For CSP, always list Variables / Domain / Constraints separately using map coloring or N-Queens, then show backtracking + MRV.

11. NLP & NLP using Generative AI

Pipeline • Embeddings • Transformer • LLM applications

11.1 What is NLP?

Natural Language Processing teaches computers to understand and produce human language (Bangla, English). It covers Natural Language Understanding (NLU) and Natural Language Generation (NLG).

11.2 Classical NLP Pipeline

Step	Task	Example
Tokenization	Split text into words/pieces	"I eat rice" → [I, eat, rice]
Stop-word removal	Drop unimportant words	the, is, and, of
Stemming / Lemmatization	Reduce to the root form	running → run, better → good
POS tagging	Grammatical category of each word	eat = Verb
NER	Detect names, places, dates	"Rahim went to Dhaka" → Person, Location
Parsing	Analyze sentence structure	Subject–Verb–Object
Vectorization	Convert to numbers: BoW, TF-IDF, Word2Vec/GloVe	"king − man + woman ≈ queen"

Tasks: sentiment analysis, text classification, machine translation, summarization, question answering, speech recognition, chatbots.

11.3 Transformers and Modern NLP

- **RNN/LSTM** read one word at a time → slow and weak on long dependencies.
- **Transformer** (2017, "Attention Is All You Need"): **self-attention** lets every word look at every other word simultaneously, so it is parallel and handles long context well.
- **BERT** (encoder, reads both directions → good for classification, NER) vs **GPT** (decoder, predicts the next token → good for text generation).

> **Self-attention, simply**
> "I sat on the river bank" vs "She deposited money at the bank": the meaning of "bank" is settled by the surrounding words (river / money). Attention computes weights for which words relate to which.

11.4 NLP with Generative AI

Before, each task needed its own model. Now one LLM does them all through a **prompt**: zero-shot or few-shot.

Task	Older approach	GenAI/LLM approach
Sentiment	Train a classifier on labeled data	Prompt: "Is this review positive or negative?"
Summarization	Extractive (pick sentences)	Abstractive (rewrite in its own words)
Translation	Statistical / Seq2Seq	LLM translation that keeps context and tone
QA / Chatbot	Rules/retrieval	LLM + **RAG** (fetch facts from your documents, then answer)

Scenario: Bangla review analysis for e-commerce

Review: "The product looks nice but delivery came very late." Prompt: "Give sentiment and aspect-wise opinion as JSON." Output: `{"product":"positive","delivery":"negative","overall":"mixed"}`. No labeled data or model training needed. The LLM can also draft an automatic reply to an upset customer.

Techniques: prompt engineering, fine-tuning (LoRA), RAG, instruction tuning, RLHF. **Challenges:** hallucination, little data for low-resource languages such as Bangla, bias, privacy. **Evaluation:** BLEU (translation), ROUGE (summarization), F1/accuracy (classification), perplexity.

Exam Tip

Show the pipeline steps on one sample sentence. Write an "NLP vs GenAI-based NLP" comparison table. BERT vs GPT differences are asked often.

12. Quantum Computing vs Generative AI

Comparison • Limitations • Working together (synergy)

12.1 Fundamental Difference

They sit at completely different layers: **quantum computing is a new kind of hardware and computation model** (*how* computation happens); **generative AI is software/algorithms that learn from data and create new content** (*what* is done). So they are not rivals but tools for different problems.

Aspect	Quantum Computing	Generative AI
Foundation	Quantum physics (qubits, superposition, entanglement)	Statistics + deep learning (Transformers, Diffusion)
Hardware	Superconducting/ion-trap, near absolute zero (~15 mK), specialized labs	Ordinary GPUs/TPUs, easily available in the cloud
Main purpose	Solve specific hard problems faster (factoring, search, simulation, optimization)	Create text, images, code, audio
Nature of output	Probabilistic, but designed to yield mathematically exact answers	Probabilistic and approximate; errors (hallucination) possible
Data needed	No large training data (algorithm-based)	Massive training data is essential
Speedup	Exponential/quadratic on some problems	Not a speedup; new capability (creativity, automation)
Maturity	Early stage: noisy, few qubits (NISQ era), error correction in progress	Widely deployed in everyday products
Problems	Decoherence, noise, scalability, cost	Hallucination, bias, huge energy use, copyright
Examples	Shor, Grover, VQE, QAOA	ChatGPT, Stable Diffusion, Copilot

Scenario: drug discovery

Quantum: simulates molecular behavior (molecules are themselves quantum systems, so a quantum computer is naturally suited). It can compute bond energies accurately, which is very costly classically.
GenAI: generates millions of possible new molecular structures (generative chemistry) and shortlists the most promising.
Together: GenAI proposes candidate molecules → quantum simulation verifies them → the best candidates go to the lab.

12.2 How They Help Each Other (Synergy)

Quantum → AI	AI → Quantum
Quantum Machine Learning (QML): potential speedups for some ML tasks (kernels, optimization, sampling)	Neural networks to decode quantum **error correction**
Quantum generative models: QGAN, quantum Boltzmann machines, QCBM	Quantum circuit design/optimization and compilers
Optimization problems in AI training (QAOA)	LLMs that write and explain quantum code (Qiskit)
Grover-based faster search/sampling	Quantum hardware calibration and noise modeling

Important reality

Today's quantum computers cannot train large GenAI models (LLMs with billions of parameters): too few qubits, too much noise, and loading large classical data into quantum states is hard. So the current approach is mainly **hybrid**: most work on classical GPUs, only specific parts on a quantum processor. Long-term potential exists with fault-tolerant quantum computers, but it is still research.

12.3 Security Angle

Quantum (Shor) can break RSA/ECC → **Post-Quantum Cryptography** (lattice-based; NIST standards such as ML-KEM/Kyber) is needed, plus the "harvest now, decrypt later" risk. **GenAI** risks: deepfakes, automatically generated phishing emails, prompt injection. Both are new challenges for cyber security (link to Topic 3 when writing).

Exam Tip (likely, since this is your teacher's own work area)

Answer structure: 1) one-line definition of each → 2) comparison table (above) → 3) drug discovery scenario → 4) synergy (QML, AI for quantum) → 5) conclusion: "Quantum computing and GenAI are complementary, not substitutes. Quantum is strong at specific hard computations; GenAI at content creation and automation; hybrid systems will be most effective in future."

Last-Minute Cheat Sheet

Read this in the final 15 minutes before the exam

#	Topic	Key points to remember
1	Quantum	Qubit $\alpha\|0\rangle+\beta\|1\rangle$; Grover $O(\sqrt{N})$, oracle + diffusion, N=4 → 1 iteration; Shor = period finding + QFT, 15=3×5 (a=7, r=4); RSA risk
2	Blockchain	Block = prev hash + Merkle root + nonce; PoW vs PoS; immutable; smart contracts; 51% attack
3	Cyber Security	CIA triad; phishing, SQLi (' OR '1'='1), DDoS, MITM; AES/RSA/SHA; MFA, firewall, IDS
4	LLM in SE	Codegen, test gen, review, repair; RAG, prompting; pass@k; hallucination, security, license
5	LLM/VLLM in DS	Text-to-SQL, cleaning, EDA; VLM = image + text (charts/OCR); vLLM = PagedAttention serving
6	CV & GenAI	Canny/SIFT/HOG; CNN, ResNet, YOLO, U-Net, ViT; GAN (G vs D), VAE, Diffusion
7	Ontology & DM	Class/Instance/Property/Axiom, OWL; KDD; Apriori: support, confidence, lift
8	KR & Frames	4 KR properties; frame = slots + facets + inheritance + defaults + demons; Vehicle → Car → MyToyota
9	SemNet, Rules, SemWeb	is-a inheritance; IF-THEN, forward vs backward chaining; RDF triple, OWL, SPARQL, layer cake
10	CBR & CSP	Retrieve-Reuse-Revise-Retain; CSP = X, D, C; map coloring; backtracking, forward checking, AC-3, MRV
11	NLP & GenAI	Tokenize → stem → POS → NER → parse; TF-IDF/Word2Vec; Transformer self-attention; BERT vs GPT; RAG
12	Quantum vs GenAI	Hardware/physics vs data/software; table; drug discovery; QML ↔ AI for quantum; hybrid; PQC

Final advice

In every answer: **Definition** → **Points** → **Example** → **Pros/Cons** → **Diagram/Table**. Good luck with your exam!